

EC341 · MID-SEM

THE PAPERS

Only one past mid-sem paper exists for this course — September 2025, the first year it ran as a core elective. It is worked out here in full, and then a **predicted paper for September 2026** that takes account of three lectures of spatial filtering the 2025 cohort had not yet been taught.

WHAT ACTUALLY GETS ASKED

1 PAPERS READ END TO END

One paper is not a pattern, so this page is honest about that. What one paper does give you is the examiner’s **style**, and that is unusually informative here: the marks are split almost exactly along the course outcomes, the questions come straight out of Gonzalez & Woods in the same order as the lectures, and every single one of them wants a calculation or a drawing rather than a description.

TOPIC	25	OF 1
2D Fourier spectrum of a sinusoid or an impulse pair		1
Convolution theorem read in the frequency domain		1
Local versus global histogram processing		1
Spatial filter masks — mean, Gaussian, Pascal		1
Bit-plane slicing and reconstruction		1
Mean and variance of a small image, two ways		1
N_4 , N_8 , N_{16} and the shortest 4-, 8-, m-paths		1
First and second derivatives of a scan line		1
Histogram matching, discrete, worked as a table		1
Continuous transformation derived from two PDFs		1

FILLED = IT APPEARED THAT YEAR · TINTED ROWS CAME UP IN EVERY PAPER WE HAVE

READ IT THIS WAY

Four questions, and the marks split by course outcome. Q1 was CO1 and worth 4; Q2, Q3 and Q4 were CO2 and worth 9, 8 and 9. That structure is the most reliable thing on the page, and it tells you where to spend revision time: **roughly 87% of the paper was CO2** — intensity transformations, histograms, and pixel relationships.

Nothing was descriptive. Every single sub-question wanted a number, a matrix, a derivation or a drawing. There was no “explain the applications of image processing”, no “list the fundamental steps”. Revise by *working* problems, not by reading.

Q2 said “attempt any three” out of four. That is a genuine gift: one whole topic can be skipped. Read all four before choosing, because the histogram-matching one in Q4 is unavoidable and the bit-plane one in Q2 takes ninety seconds.

The big change for 2026. Lectures 21, 22 and 23 — mask processing, smoothing and median filters, the Laplacian, high-boost and the gradient operators — were delivered in **September 2026**, after the 2025 paper was set. They are firmly on the syllabus now, they are the natural continuation of the Q3(b) derivative question, and they are full of exactly the small calculations this examiner likes. Our predicted paper therefore gives them real weight. If you gamble anywhere, do not gamble there.

PREDICTED PAPER — SEPTEMBER 2026

OUR READING, WITH FULL SOLUTIONS

Our prediction, not a leak. It keeps the 2025 shape exactly — four questions, 30 marks, the same CO split, “attempt any three” in Q2 — and adds the spatial-filtering material from Lectures 21–23 that the 2025 paper could not have contained. Full solutions throughout.

S.V. National Institute of Technology, Surat-7

B.Tech. – III (EC), 5th Semester

MID-SEM EXAMINATION, SEPT 2026 — *predicted*

EC-341 DIGITAL IMAGE PROCESSING (CORE ELECTIVE)

MARKS: 30

TIME: 1.5 HRS

ASSUME NECESSARY DATA WHEREVER REQUIRED

Instruction. (i) Assume necessary data wherever required. (ii) The figure to the right indicates marks.

Q-1
(a)

A 3×3 image is convolved with a 3×3 averaging mask using zero padding.
For the image below, find the output at the corner pixel (0,0) and at the pixel (0,1). C01

02

SOLUTION

image	mask (box filter)
7 9 11	1 1 1
10 50 8	1 1 1 $\times 1/9$
9 5 6	1 1 1

Zero-pad first. Write the padded array out before doing any arithmetic — most lost marks here come from taking pixels from the wrong place.

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 7 & 9 & 11 & 0 \\ 0 & 10 & 50 & 8 & 0 \\ 0 & 9 & 5 & 6 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

g(0,0) — the mask hangs off the top and the left, so five of the nine values are padded zeros:

```

window : 0  0  0
         0  7  9
         0 10 50

```

$$g(0,0) = (0+0+0+0+7+9+0+10+50) / 9 = 76 / 9 = 8.44$$

g(0,1) — only the row above is missing, so three zeros come in:

```

window : 0  0  0
         7  9 11
        10 50  8

```

$$g(0,1) = (0+0+0+7+9+11+10+50+8) / 9 = 95 / 9 = 10.56$$

$$g(0,0) = 8.44 \quad \cdot \quad g(0,1) = 10.56$$

Comment worth adding: the padding pulls the border pixels down towards zero, so a zero-padded average always darkens the edges of the image. That is the price of this method, and it is why replicating or mirroring the edge exists as an alternative.

Q-1
(b)

Find $F(u, v)$ and sketch the spectrum for (i) $f(x, y) = \delta(x, y)$ and (ii) $f(x, y) = 0.5[\delta(x-a, y) + \delta(x+a, y)]$. C01

02

SOLUTION

(i) A single impulse at the origin.

$$\begin{aligned}
 F(u, v) &= \iint \delta(x, y) e^{-j2\pi(ux+vy)} dx dy \\
 &= e^0 \\
 &= 1 \text{ for every } (u, v)
 \end{aligned}$$

$$F(u, v) = 1 - \text{constant magnitude everywhere}$$

Spectrum: a completely uniform white plane. A perfectly localised point in space contains *every* spatial frequency in equal measure — which is the same statement as “a delta function has a flat spectrum” in one dimension.

COMPUTED · TOOLS/MAKE-FIGURES.PY

A sinusoid transforms to a pair of impulses, on the axis it varies along.
The spectra are cropped to the middle 53×53 of 256×256 — everywhere else is exactly zero.

Fig.1 in space

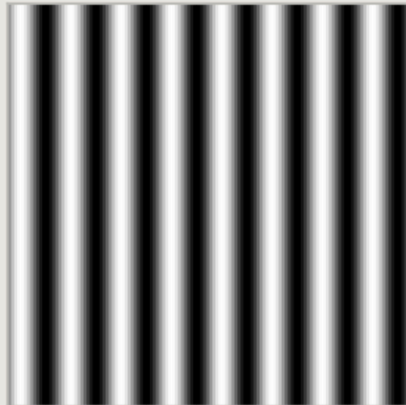
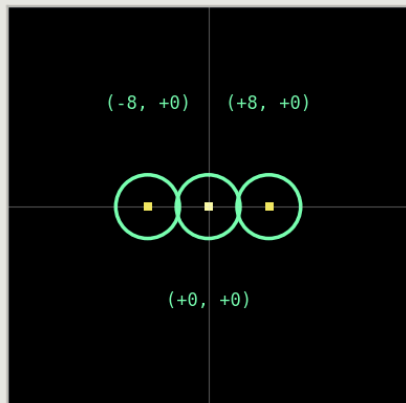


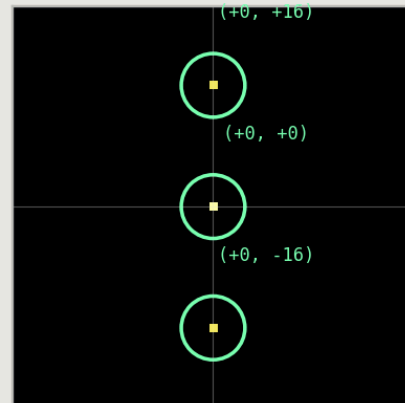
Fig.2 in space



|F1| on the u-axis, at ±8



|F2| on the v-axis, at ±16



Both spectra, computed rather than sketched: three non-zero points each and nothing anywhere else.

(ii) Two impulses, symmetric about the origin along x.

$$F(u,v) = 0.5 [e^{-j2\pi u a} + e^{+j2\pi u a}]$$

$$\text{using } e^{jT} + e^{-jT} = 2 \cos T :$$

$$\begin{aligned} F(u,v) &= 0.5 \times 2 \cos(2 \pi u a) \\ &= \cos(2 \pi u a) \end{aligned}$$

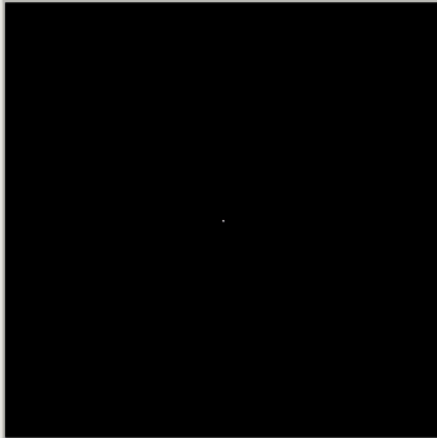
$$F(u,v) = \cos(2\pi u a) - \text{independent of } v$$

Spectrum: $|F| = |\cos(2\pi u a)|$, which is a set of **vertical bright and dark bands** running parallel to the v-axis, with the bands spaced $1/(2a)$ apart along u. The further apart the two impulses are in space, the more closely spaced the bands are in frequency — the usual reciprocal relationship.

COMPUTED · TOOLS/MAKE-FIGURES.PY

A point contains every frequency equally; a pair of points gives a cosine

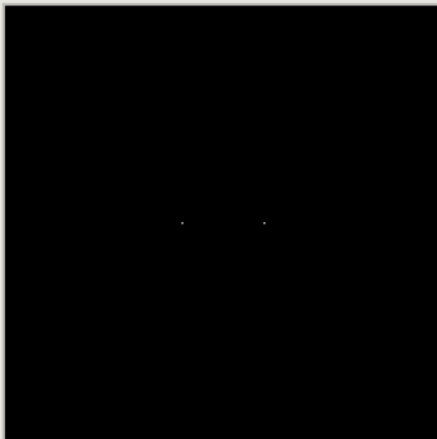
$\delta(x,y)$ one point



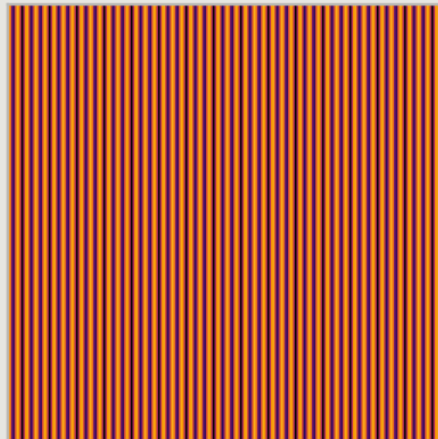
$|F| = 1$ everywhere a flat plane



$0.5[\delta(x-a,y) + \delta(x+a,y)]$



$|F| = |\cos(2\pi ua)|$ bands of period $1/(2a)$

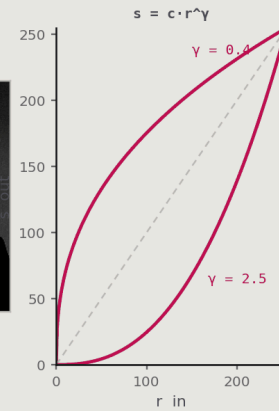


The two transforms, computed. A point gives a flat plane; a pair of points gives cosine bands of period $1/(2a)$.

THE GENERAL RULE BEHIND BOTH

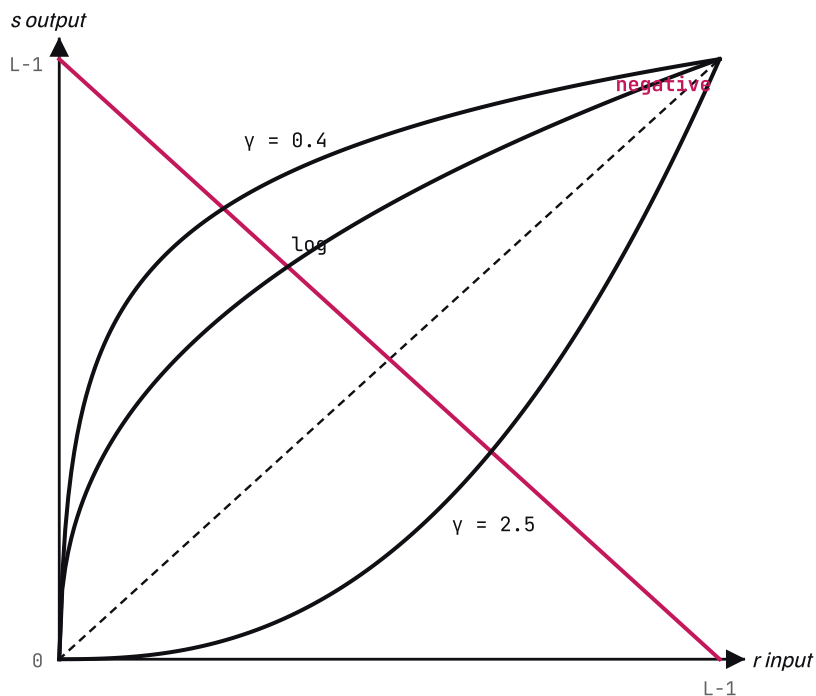
An impulse *pair* always transforms to a cosine, and a cosine always transforms to an impulse pair — the Fourier transform swaps the two. That one fact answers every question of this type, in either direction.

COMPUTED · TOOLS/MAKE-FIGURES.PY

 $\gamma = 0.4$ lifts the dark end $\gamma = 1$ the original $\gamma = 2.5$ pushes it down

The power law at three values of γ , with the curves beside it.

DRAWN FOR THE ANSWER



$\gamma < 1$ lifts the dark end, $\gamma > 1$ pushes it down.
anything above the identity line brightens; anything below darkens.

The standard transformation curves on one square. Anything above the diagonal brightens, anything below darkens.

Q-2
(a)

Attempt any three of (a) to (d). What is the primary difference between an arithmetic mean filter and a median filter? For the 3×3 window below, compute the output of each, and say which one you would use on an image corrupted by salt-and-pepper noise, and why. C02

SOLUTION

03

```

window :  18  22  33
          34 128  24
          22  19  32

```

Arithmetic mean — add all nine and divide:

$$\sum = 18 + 22 + 33 + 34 + 128 + 24 + 22 + 19 + 32 = 332$$

$$\text{mean} = \frac{332}{9} = 36.9$$

Median — sort all nine, including the centre pixel, and take the middle one:

```

sorted : 18, 19, 22, 22, [24], 32, 33, 34, 128
                  ^ the 5th of 9

```

```

median = 24

```

mean = 36.9 · median = 24

The primary difference. The mean filter is **linear** — it multiplies and adds, so every input, however wrong, contributes to the output. The median filter is an **order-statistic** filter and therefore **non-linear** — it ranks the values and picks one, so an extreme value is never selected.

For salt-and-pepper noise, use the median. Two reasons, and both are needed for full marks:

- The 128 here is a noise spike. The mean lets it drag the answer up to 36.9 — and worse, that single bad pixel contributes to *all nine* of the output pixels around it, so one speck becomes a gray smudge.
- The median simply never chooses it. The noise pixel is **removed outright** and replaced by a genuine neighbouring value, so at the same mask size you get far better noise reduction with **considerably less blurring**.

COMPUTED · TOOLS/MAKE-FIGURES.PY

Mean error against the clean image — mean 15.0, Gaussian 14.3, median 6.2. The median does not average the spike in, it refuses to pick it.

salt and pepper, 8% of pixels



mean 3x3



Gaussian 5x5 (Pascal)



median 3x3



Salt-and-pepper noise and the three smoothers. Mean absolute error against the clean image: mean 15.0, Gaussian 14.3, median 6.2.

Q-2
(b)

Derive the 3×3 Laplacian mask from its definition. Explain why the Laplacian alone is not an enhanced image, and write the single mask that sharpens an image in one pass. C02

03

SOLUTION

Definition.

$$\text{Laplacian : } \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Each second derivative as a difference:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

Add them:

$$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Read the coefficients straight off — +1 on each of the four side neighbours, −4 on the centre, 0 on the corners:

$$\begin{array}{ccc} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{array} \quad \text{coefficients sum to 0}$$

Why it is not an enhanced image. The Laplacian is a pure derivative operator, so it responds only to *change*. Every region of slowly varying gray gives zero. The result is therefore edges and fine detail sitting on a **dark, featureless background** — a map of the discontinuities, not a picture.

The one-pass sharpening mask. Add the edges back onto the original:

$$\begin{aligned}
 g(x, y) &= f(x, y) - \nabla^2 f(x, y) \\
 &\quad \text{centre of the mask is negative} \\
 &= f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)] \\
 &= 5f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)]
 \end{aligned}$$

$$\begin{array}{ccc}
 0 & -1 & 0 \\
 -1 & 5 & -1 \\
 0 & -1 & 0
 \end{array}
 \quad \text{coefficients sum to 1}$$

Composite mask $[0 \ -1 \ 0; \ -1 \ 5 \ -1; \ 0 \ -1 \ 0]$

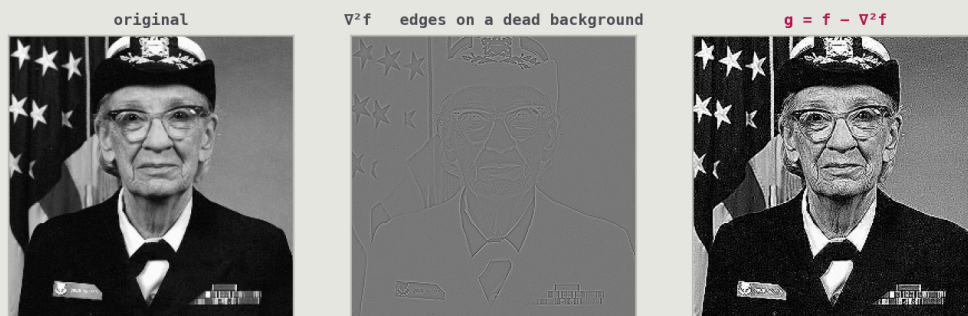
THE CHECK THAT CATCHES SIGN ERRORS

A **pure derivative** mask has coefficients summing to **0**, so it gives no response on a flat region. A **sharpening** mask has coefficients summing to **1**, because it is “the original, plus its edges” and must preserve average brightness. Add up any mask you write and you will know at once which one you have built.

And the sign pairing: centre $-4 \rightarrow$ use $g = f - \nabla^2 f$. Centre $+4 \rightarrow$ use $g = f + \nabla^2 f$. Say which convention you are using before you start.

COMPUTED · TOOLS/MAKE-FIGURES.PY

The Laplacian on its own is not an image — it is a map of the discontinuities. Add it back and you have the sharpened picture.



The Laplacian, and the one-pass sharpened result.

DRAWN FOR THE ANSWER

Box / mean

1	1	1
1	1	1
1	1	1

 $1/9$

smooths, blurs edges too

Gaussian

1	2	1
2	4	2
1	2	1

 $1/16$

smooths, weights the centre

Laplacian

0	1	0
1	-4	1
0	1	0

sum 0

a pure second derivative

Sharpen

0	-1	0
-1	5	-1
0	-1	0

sum 1

the original plus its edges

the fastest check there is:

coefficients sum to 1 → a smoothing or sharpening mask, brightness is preserved

coefficients sum to 0 → a derivative mask, flat regions give zero response

The Laplacian and the one-pass sharpening mask beside the two smoothing masks. Add up the coefficients: 0 means a derivative, 1 means a sharpener.

Q-2
(c)

For the 3×3 window below, compute the gradient magnitude at the centre pixel using the Sobel operator and the $|G_x| + |G_y|$ approximation. State what the two components tell you about the direction of the edge. C02

03

SOLUTION

```

window :  z1 z2 z3      10  10  10
          z4 z5 z6    =  10  10  10
          z7 z8 z9      50  50  50

```

The Sobel masks:

```

Gx =  -1  -2  -1      Gy =  -1  0  1
      0   0   0        -2  0  2
      1   2   1        -1  0  1

```

$$\begin{aligned}
 Gx &= (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) \\
 &= (50 + 100 + 50) - (10 + 20 + 10) \\
 &= 200 - 40 \\
 &= 160
 \end{aligned}$$

$$\begin{aligned}
 Gy &= (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) \\
 &= (10 + 20 + 50) - (10 + 20 + 50) \\
 &= 80 - 80 \\
 &= 0
 \end{aligned}$$

$$M(x,y) \sim |G_x| + |G_y| = 160 + 0 = 160$$

Gradient magnitude ≈ 160

What the components say. G_x is large and G_y is exactly zero, so the brightness is changing rapidly **down the columns** and not at all **across the rows**. The gradient vector therefore points straight down, and since the gradient points *across* an edge, the edge itself runs **horizontally** — which is exactly what the window shows: a flat region of 10s sitting on a flat region of 50s.

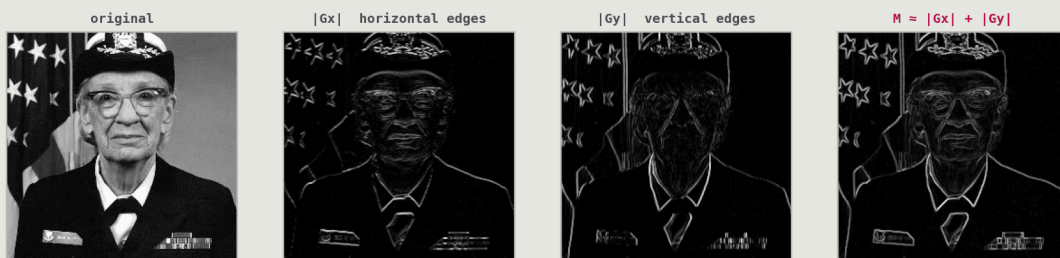
TWO THINGS TO ADD FOR THE THIRD MARK

Why the 2. The middle coefficient of each row is ± 2 to give more weight to the centre point, which smooths as well as differentiates. That is the only difference between Sobel and Prewitt, and it makes Sobel less sensitive to noise.

The free check. Every gradient mask's coefficients sum to zero, so it gives a response of 0 in an area of constant gray. Add up either mask above to confirm you have copied it correctly.

COMPUTED · TOOLS/MAKE-FIGURES.PY

G_x answers to change down the columns, G_y to change across the rows — and the gradient points across an edge, never along it.



Sobel on a real image: $|G_x|$ finds horizontal edges, $|G_y|$ vertical.

DRAWN FOR THE ANSWER

the window

10	10	10
10	10	10
50	50	50

the edge runs this way

the gradient points this way

G_h is zero, so nothing is changing across the rows. All the change is down the columns, so the gradient points down – and since it points across an edge, the edge itself is horizontal.

SOBEL

G_x

-1	-2	-1
0	0	0
1	2	1

= 160

G_h

-1	0	1
-2	0	2
-1	0	1

= 0

$M \approx |G_x| + |G_h| = 160 + 0 = 160$

The Sobel pair at a horizontal edge. All the change is down the columns, so G_h is zero and the edge runs across the window.

Q-2
(d)

Using the concept of image reconstruction from n-bit planes, find the new value of each of the pixels 200 and 75 when only the 8th and 7th bit planes are retained. C02

03

SOLUTION

Write each value in 8-bit binary and label the planes. The 8th bit plane is the MSB (weight 128) and the 7th is the next one down (weight 64).

bit :	8	7	6	5	4	3	2	1	
weight:	128	64	32	16	8	4	2	1	
200 =	1	1	0	0	1	0	0	0	128 + 64 + 8 = 200 check OK
75 =	0	1	0	0	1	0	1	1	64 + 8 + 2 + 1 = 75 check OK

Keep only bits 8 and 7, set the rest to zero:

200 →	1	1	0	0	0	0	0	0	0	= 128 + 64 = 192
75 →	0	1	0	0	0	0	0	0	0	= 0 + 64 = 64

200 → 192 · 75 → 64

The comment that earns the last mark. Two planes out of eight can only produce **four** distinct output values — 0, 64, 128 and 192 — so the reconstruction is a coarse four-level version of the image. The large-scale shapes survive, because

most of a pixel's value lives in its top bits, but all fine shading is gone. That is exactly why the higher-order planes are said to carry the visually significant data and the lower ones the subtle detail.

Q-3
(a)

For the image segment below, with $V = \{1, 2\}$: (i) write the coordinates of $N_4(p)$, $N_D(p)$ and $N_8(p)$. (ii) compute the lengths of the shortest 4-, 8- and m-paths between p and q . If a particular path does not exist, explain why. C02

04

SOLUTION

col:	0	1	2	3
row 0:	3	1	2	1(q)
row 1:	2	2	0	2
row 2:	1	2	1	1
row 3:	1(p)	0	1	2

(i) The neighbourhoods of $p = (3,0)$. These are positional sets and do not depend on V :

$$\begin{aligned} N_4(p) &= (2, 0), (4, 0), (3, -1), (3, 1) \\ &= (2, 0), (3, 1) \\ &\text{the others are outside the image} \end{aligned}$$

$$\begin{aligned} N_D(p) &= (2, -1), (2, 1), (4, -1), (4, 1) \\ &= (2, 1) \\ &\text{the others are outside the image} \end{aligned}$$

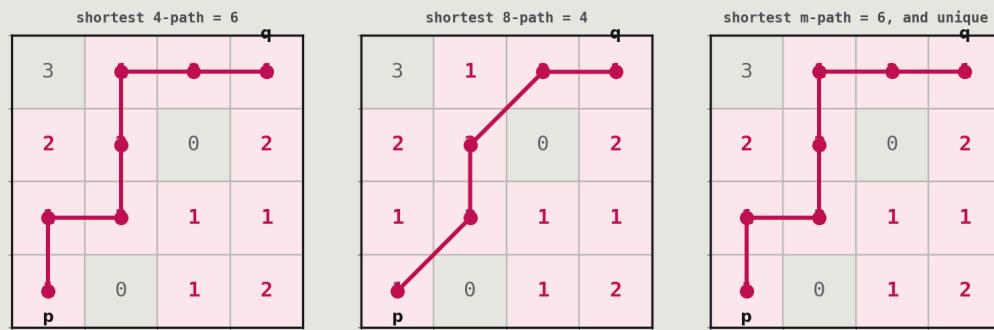
$$N_8(p) = N_4(p) \cup N_D(p) = (2, 0), (3, 1), (2, 1)$$

Always say this: p is a border pixel, so several of its neighbours fall outside the image and it has fewer than eight. Examiners look for that sentence.

(ii) Paths from $p = (3,0)$ to $q = (0,3)$, with $V = \{1,2\}$. First mark which pixels are in V :

COMPUTED · TOOLS/MAKE-FIGURES.PY

$V = \{1, 2\}$. Every length below was found by breadth-first search over this exact array, with the m-adjacency rule applied at each diagonal step.



The grid from the question with $V = \{1, 2\}$, and all three shortest paths drawn. Every length was found by breadth-first search over this exact array, with the m-adjacency rule applied at each diagonal step — the 8-path is 4, and both the 4-path and the m-path are 6.

Shortest 4-path. Only side steps are allowed, and every pixel on the way must be in V :

$(3,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (1,1) \rightarrow (1,0)$? no, going backwards

take instead:

$(3,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (1,1) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3)$

check each: $(2,0)=1$ OK, $(2,1)=2$ OK, $(1,1)=2$ OK,
 $(0,1)=1$ OK, $(0,2)=2$ OK, $(0,3)=1$ OK

length = 6 steps

The city-block lower bound is $D_4 = |3-0| + |0-3| = 6$, so 6 is optimal.

Shortest 4-path = 6

Shortest 8-path. Diagonals now count as one step:

$(3,0) \rightarrow (2,1) \rightarrow (1,1)$? that is a side step... better:

$(3,0) \rightarrow (2,1) \rightarrow (1,2)$? $(1,2) = 0$, NOT in V

$(3,0) \rightarrow (2,1) \rightarrow (1,1) \rightarrow (0,2) \rightarrow (0,3)$
 diag side diag side

check: $(2,1)=2$ OK, $(1,1)=2$ OK, $(0,2)=2$ OK, $(0,3)=1$ OK

length = 4 steps

The chessboard lower bound is $D_8 = \max(3, 3) = 3$, so we should check whether 3 is reachable. A 3-step path needs three diagonal moves: $(3,0) \rightarrow (2,1) \rightarrow (1,2) \rightarrow (0,3)$. But $(1,2) = 0$, which is not in V , so that route is blocked and 4 is the best available.

Shortest 8-path = 4

Shortest m-path. Test each diagonal step of the 8-path against the m-adjacency rule — a diagonal step is allowed only if $N_4(p) \cap N_4(q)$ contains **no** pixel from V .

```

step (3,0) → (2,1) is diagonal.
  N4(3,0) = {(2,0), (3,1)}           [inside the image]
  N4(2,1) = {(1,1), (3,1), (2,0), (2,2)}
  common  = {(2,0), (3,1)}
  values  : (2,0) = 1 → IS in V

  so the diagonal is REFUSED; a route round already exists.

go round instead: (3,0) → (2,0) → (2,1)           [two side steps]

step (1,1) → (0,2) is diagonal.
  N4(1,1) = {(0,1), (2,1), (1,0), (1,2)}
  N4(0,2) = {(0,1), (0,3), (1,2)}
  common  = {(0,1), (1,2)}
  values  : (0,1) = 1 → IS in V

  refused again; go round: (1,1) → (0,1) → (0,2)

```

m-path : $(3,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (1,1) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3)$
 length = 6 steps

Shortest m-path = 6, and it is unique

THE COMPARISON THAT FINISHES THE ANSWER

4-path 6 · 8-path 4 · m-path 6. The m-path can never be shorter than the 8-path, because m-adjacency only *ever removes* links from 8-adjacency, never adds them. What it buys in exchange is that the path is **unique** — with 8-adjacency there were several 4-step routes and no way to call one of them “the” path, which is the whole reason m-adjacency was invented.

DRAWN FOR THE ANSWER

 D_4 - CITY BLOCK

4	3	2	3	4
3	2	1	2	3
2	1	0	1	2
3	2	1	2	3
4	3	2	3	4

 D_8 - CHESSBOARD

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

EUCLIDEAN

2.8	2.2	2.0	2.2	2.8
2.2	1.4	1.0	1.4	2.2
2.0	1.0	0.0	1.0	2.0
2.2	1.4	1.0	1.4	2.2
2.8	2.2	2.0	2.2	2.8

$D_4 = |x-s| + |y-t|$ - contours are diamonds. The pixels at $D_4 \leq 1$ are exactly $N_4(p)$.

$D_8 = \max(|x-s|, |y-t|)$ - contours are squares. The pixels at $D_8 \leq 1$ are exactly $N_8(p)$.

Euclidean is the true distance, but it is not an integer, which is why the other two exist.

D_4 , D_8 and the true Euclidean distance on the same patch. The pixels at $D_4 \leq 1$ are exactly $N_4(p)$, and those at $D_8 \leq 1$ are exactly $N_8(p)$.

Q-3
(b)

The horizontal gray-level profile of a scan line is given below. Find the first and second derivatives, and comment on what each one does at the ramp, at the isolated point and at the step. Show the computation from the second through to the penultimate point. C02

04

SOLUTION

scan line: 7 7 7 6 5 4 3 3 3 3 7 3 3 3 3

First derivative $f'(x) = f(x+1) - f(x)$, taken from the second sample to the penultimate one:

x	:	1	2	3	4	5	6	7	8	9	10	11	12	13
f(x)	:	7	7	6	5	4	3	3	3	3	7	3	3	3
f'(x)	:	0	-1	-1	-1	-1	0	0	0	4	-4	0	0	0

Second derivative $f''(x) = f(x+1) + f(x-1) - 2f(x)$, over the same range:

x	:	1	2	3	4	5	6	7	8	9	10	11	12	13
f''(x)	:	0	-1	0	0	0	1	0	0	4	-8	4	0	0

Comments — the part the marks are actually for.

FEATURE	FIRST DERIVATIVE	SECOND DERIVATIVE
Constant regions (7 7 7 and 3 3 3)	zero	zero
Along the ramp (7→3)	Non-zero all the way , a steady -1 , so it produces a thick response spanning the whole ramp	Zero in the middle of the ramp; non-zero only at the two ends, so a thin response marking just the corners
Isolated point (the single 7)	$+4$ then -4	$+4, -8, +4$ — a much stronger, sharper response
Zero crossing	none	Yes — the sign change between the -8 and the $+4$ pins the feature to the exact pixel

The conclusion: first derivatives give thicker edges and respond to gradual change; second derivatives respond far more strongly to fine detail such as isolated points and thin lines, and their zero crossings locate an edge precisely. That is why the Laplacian — a second derivative — is the standard sharpening filter, while the gradient — a first derivative — is the standard edge detector.

Q-4
(a)

A 4×4 image with $L = 8$ has the pixel values shown. Perform histogram equalisation. Give the transformation table, the output image, and sketch both the input and the output histograms. C02

05

SOLUTION

```
image :  1  3  3  2
         4  2  4  3
         3  2  3  5
         2  4  2  4      MN = 16,  L = 8,  L-1 = 7
```

Step 1 — build the histogram. Count each gray level:

R_K	0	1	2	3	4	5	6	7
N_K	0	1	5	5	4	1	0	0

Check: $0+1+5+5+4+1+0+0 = 16 \checkmark$

Step 2 — the running sum and the transformation.

$$s_k = (L - 1) \times \left(\frac{1}{MN}\right) \times \sum_{j=0 \dots k} n_j$$
$$= \frac{7 \times (\text{cumulative count})}{16}$$

R _k	N _k	CUMULATIVE	S _k = 7×CUM/16	ROUNDED
0	0	0	0.0000	0
1	1	1	0.4375	0
2	5	6	2.6250	3
3	5	11	4.8125	5
4	4	15	6.5625	7
5	1	16	7.0000	7
6	0	16	7.0000	7
7	0	16	7.0000	7

Step 3 — the mapping and the output image.

1 → 0 2 → 3 3 → 5 4 → 7 5 → 7

input

1 3 3 2

4 2 4 3

3 2 3 5

2 4 2 4

output

0 5 5 3

→ 7 3 7 5

5 3 5 7

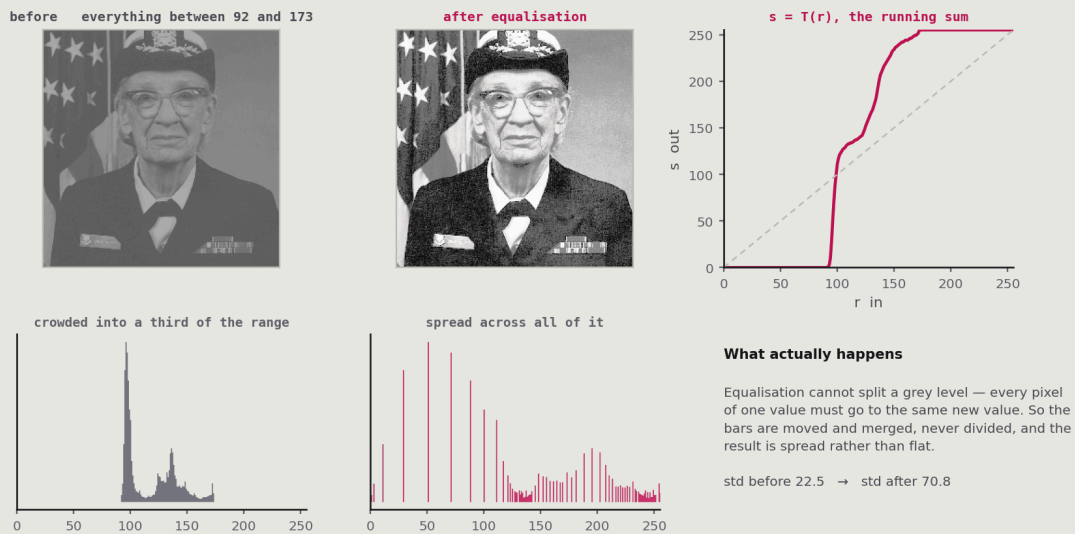
3 7 3 7

Step 4 — the output histogram.

S	0	1	2	3	4	5	6	7
COUNT	1	0	0	5	0	5	0	5

Check: 1+5+5+5 = 16 ✓

COMPUTED · TOOLS/MAKE-FIGURES.PY

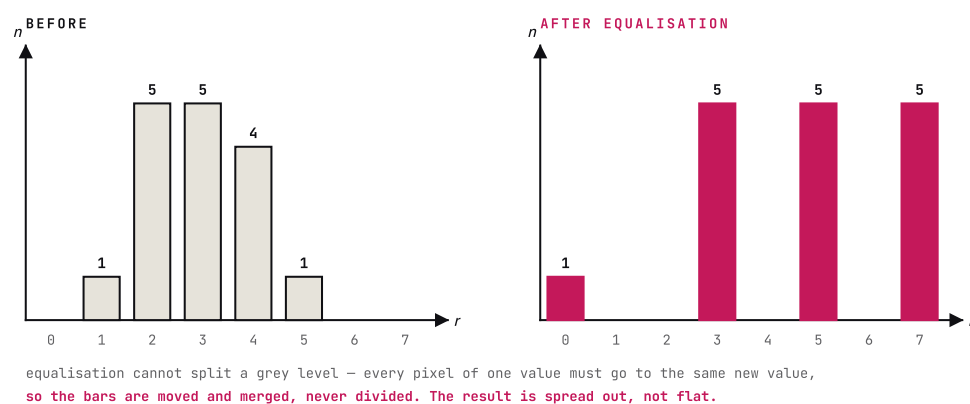


Equalisation on a real image, with both histograms and the transfer curve that does the work.

THE COMMENT THAT EARNS THE LAST MARK

The output histogram is **not flat**, and it never will be in the discrete case. Equalisation cannot split a gray level — all five pixels of value 2 must go to the *same* new value — so the bars can only be **moved and merged**, never divided. What it does achieve is **spreading them over the full range**, which is where the contrast improvement comes from. Only in the continuous case is the result exactly uniform.

DRAWN FOR THE ANSWER



The 16 pixels before and after. Three bars, three positions — spread across the range instead of crowded into the middle.

Q-4 (b) An image has the intensity PDF $p_r(r) = 2r/(L-1)^2$ for $0 \leq r \leq L-1$. Derive the transformation that will produce an image whose intensity PDF is $p_z(z) = 3z^2/(L-1)^3$. Verify the result at $r = 0$ and $r = L-1$. C02

SOLUTION

The method for histogram matching in the continuous case is always the same **three steps**: equalise the input, equalise the target, then set the two equal and solve for z .

Step 1 — equalise the input.

$$\begin{aligned}
 s &= T(r) = (L-1) \int_0^r p_r(w) dw \\
 &= (L-1) \int_0^r \frac{2w}{(L-1)^2} dw \\
 &= (L-1) \times \left[\frac{w^2}{(L-1)^2} \right] \text{ from } 0 \text{ to } r \\
 &= \frac{(L-1) \times r^2}{(L-1)^2} \\
 T(r) &= \frac{r^2}{L-1}
 \end{aligned}$$

Step 2 — equalise the target.

$$\begin{aligned}
 G(z) &= (L-1) \int_0^z p_z(t) dt \\
 &= (L-1) \int_0^z \frac{3t^2}{(L-1)^3} dt \\
 &= (L-1) \times \left[\frac{t^3}{(L-1)^3} \right] \text{ from } 0 \text{ to } z \\
 &= \frac{(L-1) \times z^3}{(L-1)^3} \\
 G(z) &= \frac{z^3}{(L-1)^2}
 \end{aligned}$$

Step 3 — set $G(z) = T(r)$ and solve for z . Both images, after equalisation, must have the same uniform distribution, so their transformed values must match:

$$\begin{aligned}\frac{z^3}{(L-1)^2} &= \frac{r^2}{L-1} \\ z^3 &= r^2(L-1) \\ z &= [r^2(L-1)]^{\frac{1}{3}}\end{aligned}$$

$$z = \sqrt[3]{[r^2(L-1)]}$$

Verification at the two ends — a monotonic mapping of the full range must fix both endpoints:

$$\begin{aligned}r = 0: \quad z &= [0 \times (L-1)]^{\frac{1}{3}} = 0 \text{ OK} \\ r = L-1: \quad z &= [(L-1)^2(L-1)]^{\frac{1}{3}} \\ &= [(L-1)^3]^{\frac{1}{3}} \\ &= L-1 \text{ OK}\end{aligned}$$

A worked value, for an 8-bit image where $L-1 = 255$:

$$\begin{aligned}r = 128 &\rightarrow z = [128^2 \times 255]^{\frac{1}{3}} \\ &= [16384 \times 255]^{\frac{1}{3}} \\ &= (4,177,920)^{\frac{1}{3}} \\ &= 161.0\end{aligned}$$

So mid-grey is pushed *up* to 161. That is the right direction: the target PDF $3z^2/(L-1)^3$ grows as z^2 , so it wants more pixels at the bright end than the input's linear PDF supplies, and the transformation obliges by stretching values upwards.

WHY YOU NEVER INVERT G DIRECTLY

The formal answer is $z = G^{-1}[T(r)]$, and it is tempting to find G^{-1} first. Do not. Setting $G(z) = T(r)$ and solving the resulting algebra for z is the same thing and is almost always easier — here it is a single cube root instead of an inverse function.

MID-SEM, SEPTEMBER 2025

EC-341 · 30 MARKS · THE ONLY PAST PAPER

The first mid-sem for this course. Four questions, marks split by course outcome, and not a single descriptive sub-question — everything wants a number, a matrix or a drawing.

S.V. National Institute of Technology, Surat-7

B.Tech. – III (EC), 5th Semester

MID-SEM EXAMINATION, SEPT 2025

EC-341 DIGITAL IMAGE PROCESSING (CORE ELECTIVE)

MARKS: 30

TIME: 1.5 HRS

ASSUME NECESSARY DATA WHEREVER REQUIRED

Q-1
(a)

(i) Draw the spectrum of Fig.1 and Fig.2. (ii) Draw the spectrum that results from the convolution of the two images shown in Fig.1 and Fig.2. **Fig.1:** a horizontal sine wave of frequency 8. **Fig.2:** a vertical sine wave of frequency 16. C01

02

FROM THE PAPER

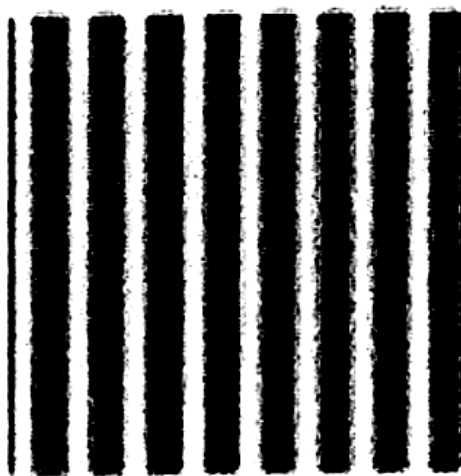
**Fig.1****Fig.2**

Fig.1 and Fig.2 exactly as they are printed on the paper — a horizontal sine of frequency 8, and a vertical sine of frequency 16.

SOLUTION

(i) The two spectra.

Fig.1 is a sine wave varying along x with 8 cycles across the image — it looks like *vertical* stripes. A sinusoid transforms to a **pair of impulses**, placed at its frequency on the axis it varies along:

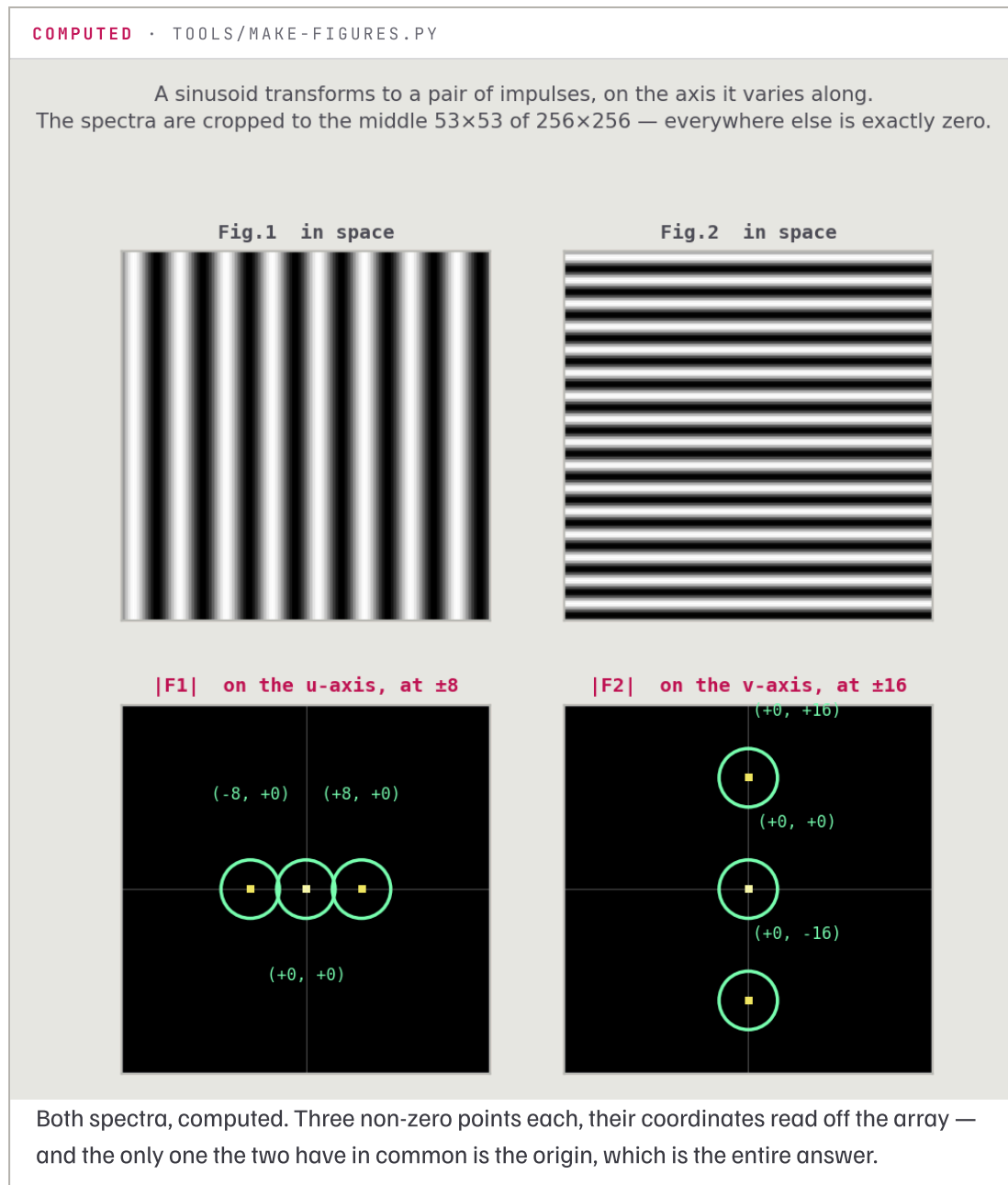


Fig.2 varies along y with 16 cycles — *horizontal* stripes. Same rule, other axis:

```
F2(u,v) : impulses at (u,v) = (0, +16) and (0, -16), plus DC

      v
      .      two bright points on the v-axis, 16 units either side
      |
-----+-----> u   (further out than Fig.1, because 16 > 8)
      |
      .
```

(ii) **The spectrum of the convolution.** Use the convolution theorem — convolution in space is **multiplication** in frequency:

$$f_1 ** f_2 \longleftrightarrow F_1(u, v) \times F_2(u, v)$$

Now look at where each one is non-zero:

F_1 is non-zero only at $(0, 0), (+8, 0), (-8, 0)$	on the u-axis
F_2 is non-zero only at $(0, 0), (0, +16), (0, -16)$	on the v-axis

the product is non-zero only where BOTH are non-zero,
and the only point they share is the origin.

The result is a single impulse at the origin — the convolved image is a uniform grey field

Every stripe pattern has vanished. The DC terms of the two images multiply to give the constant, and every other frequency component is killed because the other image has nothing at that frequency to multiply it by.

THE CLEANER WAY TO SAY IT

Convolving with a pure sinusoid is a **filter** that passes only the frequency of that sinusoid. Fig.2 contains no energy at $u = \pm 8$, so Fig.1's stripes are removed entirely; and Fig.1 contains no energy at $v = \pm 16$, so Fig.2's stripes are removed too. Nothing is left but the average brightness.

If the two sinusoids had been at the *same* frequency and on the *same* axis, the answer would be completely different — the impulses would coincide and the stripes would survive, amplified.

COMPUTED · TOOLS/MAKE-FIGURES.PY

The two images, generated from $\sin(2\pi u x/N)$

Fig.1 horizontal sine, frequency 8

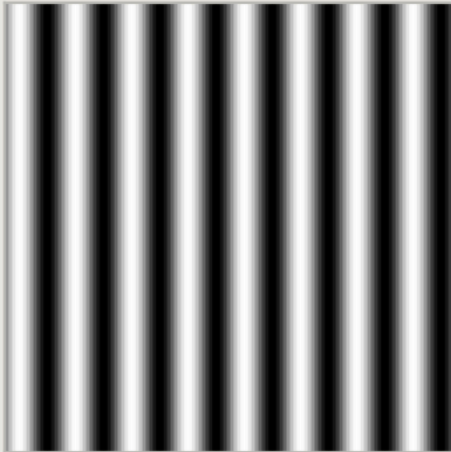


Fig.2 vertical sine, frequency 16



The two images, generated from the description on the paper rather than traced off the scan.

COMPUTED · TOOLS/MAKE-FIGURES.PY

A sinusoid transforms to a pair of impulses, on the axis it varies along.
The spectra are cropped to the middle 53×53 of 256×256 — everywhere else is exactly zero.

Fig.1 in space

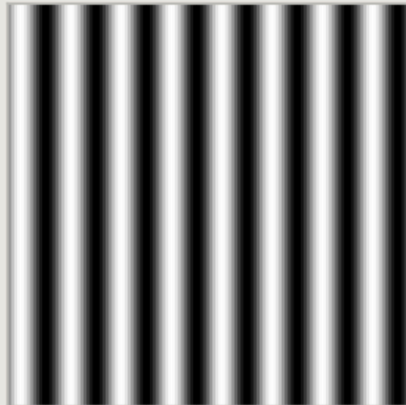
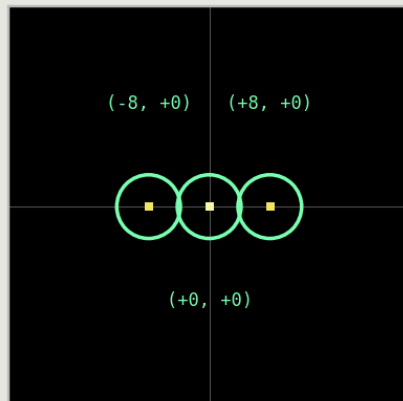
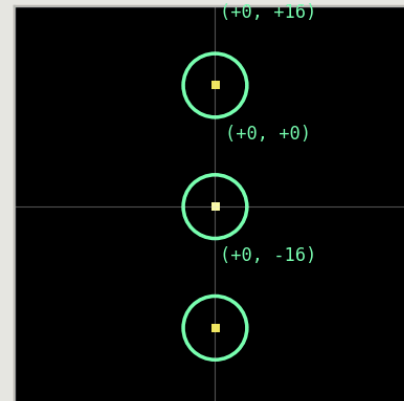


Fig.2 in space

 $|F1|$ on the u-axis, at ± 8  $|F2|$ on the v-axis, at ± 16 

Their spectra, from `np.fft.fft2`. Three non-zero points each and nothing anywhere else — which is the whole answer to part (i).

COMPUTED · TOOLS/MAKE-FIGURES.PY

The product is non-zero only where both spectra are — and the only point they share is the origin. Range of the result: 0

Fig.1

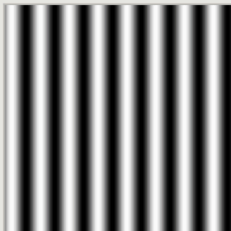
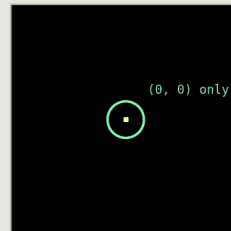
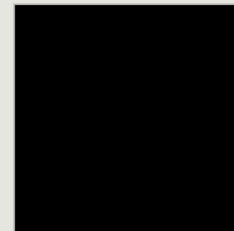


Fig.2

 $F1 \times F2$ one point, at the origin

the convolution



And part (ii), carried out. The product of the two spectra is non-zero at the origin alone, so the convolution is a uniform field. The measured range printed on the figure is effectively zero.

Q-1
(b)

Find $F(u,v)$ for the following and also draw the spectrum of it. (i) $f(x,y) = \delta(x,y)$
(ii) $f(x,y) = 0.5[\delta(x, y-a) + \delta(x, y+a)]$ C01

02

SOLUTION

(i) $f(x,y) = \delta(x,y)$ — a single impulse at the origin.

$$F(u,v) = \int \int \delta(x,y) e^{-j2\pi(ux+vy)} dx dy$$

the sifting property picks out the value at $x = y = 0$:

$$F(u,v) = e^{-j2\pi(0)} = 1 \quad \text{for all } u \text{ and } v$$

$F(u,v) = 1$ — flat, constant magnitude everywhere

Spectrum: a uniform white plane, no structure at all. A perfectly localised point contains every spatial frequency equally.

(ii) $f(x,y) = 0.5[\delta(x, y-a) + \delta(x, y+a)]$ — two impulses at $(0, a)$ and $(0, -a)$, symmetric about the origin along the y -axis.

$$F(u,v) = 0.5 [e^{-j2\pi v a} + e^{+j2\pi v a}]$$

using $e^{jT} + e^{-jT} = 2 \cos T$:

$$F(u,v) = 0.5 \times 2 \cos(2\pi v a)$$

$$F(u,v) = \cos(2\pi v a) \quad \text{-- independent of } u$$

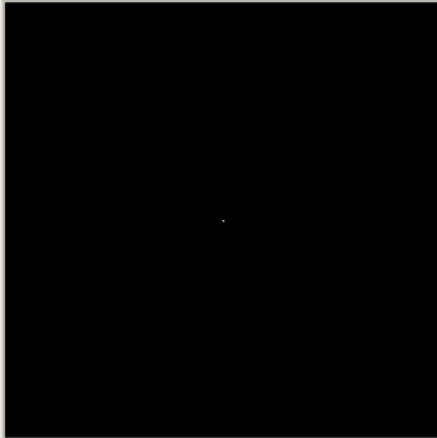
$F(u,v) = \cos(2\pi v a)$

Spectrum: $|F| = |\cos(2\pi v a)|$ — a set of **horizontal bright and dark bands**, running parallel to the u -axis and repeating every $1/(2a)$ along v .

COMPUTED · TOOLS/MAKE-FIGURES.PY

A point contains every frequency equally; a pair of points gives a cosine

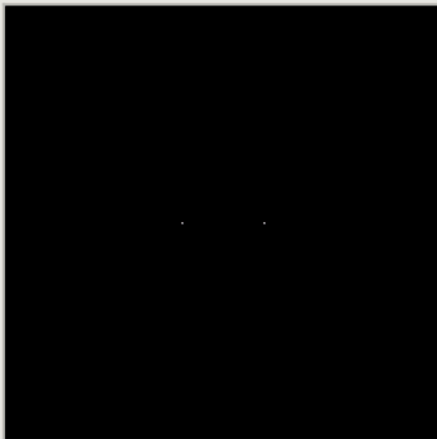
$\delta(x,y)$ one point



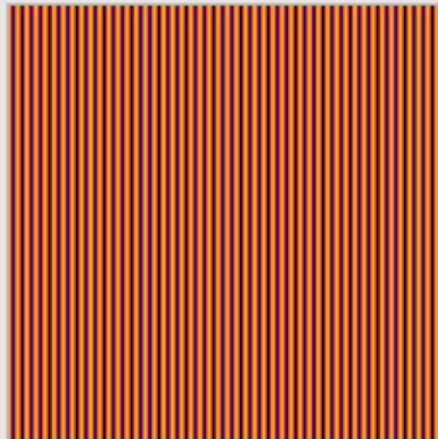
$|F| = 1$ everywhere a flat plane



$0.5[\delta(x-a,y) + \delta(x+a,y)]$



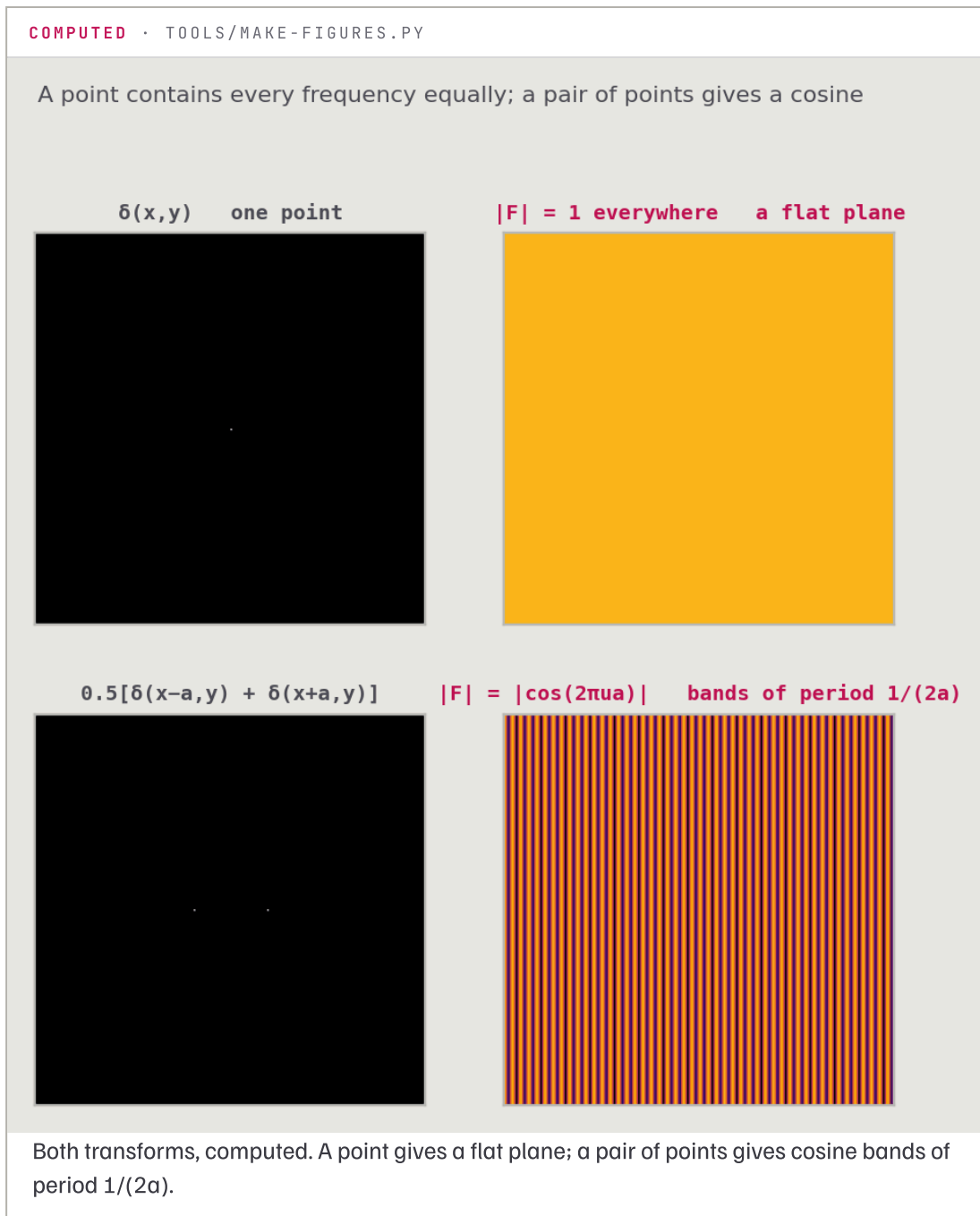
$|F| = |\cos(2\pi u a)|$ bands of period $1/(2a)$



The pair of impulses and its transform, computed: $|\cos(2\pi u a)|$, which is bands running parallel to the u -axis with period $1/(2a)$.

THE RULE BEHIND BOTH PARTS

An **impulse pair transforms to a cosine**, and a **cosine transforms to an impulse pair**. Note also the reciprocal relationship: the further apart the impulses are in space (larger a), the more closely spaced the bands are in frequency. And part (i) is just the limiting case $a = 0$, where the two impulses merge and the cosine flattens to the constant 1.



Q-2
(a)

Attempt any three of (a) to (d). (i) Normally, histogram equalization is a global process; explain how it can be used for local enhancement. (ii) When performing local histogram equalization, why use the sliding window method? (iii) Why use the block method? C02

03

SOLUTION

(i) From global to local. Global equalisation builds **one** transformation function from the gray-level content of the *entire* image and applies it to every pixel. That fails when the detail you care about lives in a small area — a few hundred pixels

cannot move a histogram built from a quarter of a million, so a locally dark region stays dark.

The fix is to do exactly the same work in a small window:

1. Define a neighbourhood — a square window is usual.
2. Compute the histogram of the pixels **inside that window only**.
3. Equalise it, and apply the resulting transformation to **the centre pixel alone**.
4. Move the window to the next pixel and repeat.

(ii) Why the sliding window method.

- **A new transformation for every pixel**, so the enhancement adapts continuously to the local content — this is the version that reveals detail hidden in dark corners.
- **No blocking artefacts**. Neighbouring pixels have nearly identical windows, so their transformations are nearly identical and the output varies smoothly.
- **It is cheaper than it looks**. When the window moves one column, only one column of pixels leaves the histogram and one enters, so the histogram is updated **incrementally** rather than rebuilt — which is what makes the method practical at all.

(iii) Why the block method.

- **Speed**. The image is divided into non-overlapping blocks and each block is equalised once. That is one transformation per *block* instead of one per *pixel* — orders of magnitude less work.
- **Simplicity**, and it parallelises trivially since the blocks are independent.

The price is **visible blocking artefacts**: adjacent blocks get unrelated transformations, so a discontinuity appears at every block boundary. The usual compromise is to equalise per block and then **interpolate** between the neighbouring blocks' transformations — which is exactly what CLAHE does.

Q-2
(b)

What is the primary difference in the results from an arithmetic mean compared to a Gaussian spatial filter? Generate the 5×5 Gaussian filter mask using Pascal's triangle. C02

03

SOLUTION

The primary difference. Both are smoothing filters and both reduce noise, but:

	ARITHMETIC MEAN (BOX FILTER)	GAUSSIAN
Weights	All equal — a pixel at the corner of the window counts as much as the centre one	Centre heaviest, falling off smoothly with distance
Blurring	More — distant pixels drag the centre value a long way	Less , for the same amount of noise reduction
Artefacts	The mask has sharp edges, so its frequency response is a sinc with sidelobes → ringing and a boxy look	The transform of a Gaussian is a Gaussian — no sidelobes, no ringing
Shape	Square — it smooths more along the diagonals than along the axes	Isotropic — circularly symmetric, direction-independent

In one sentence: for the same window size, the Gaussian gives a smoother, more natural result with better-preserved edges and no ringing, because it weights the centre most and falls off gradually instead of stopping abruptly.

Generating the 5×5 mask from Pascal's triangle. The binomial coefficients are a discrete approximation to a Gaussian, so the row of Pascal's triangle with five entries is the 1-D kernel:

Pascal's triangle:

```

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1    ← five entries, the row we need

```

The 2-D mask is the **outer product** of that row with itself — which works because a Gaussian is separable:

$$\begin{array}{cccccc}
 & & & & & [1] \\
 & & & & & [4] \\
 [6] \times [1 & 4 & 6 & 4 & 1] \\
 & & & & & [4] \\
 & & & & & [1]
 \end{array}$$

$$\begin{array}{rcccccc}
 & & 1 & 4 & 6 & 4 & 1 \\
 & & 4 & 16 & 24 & 16 & 4 \\
 1/256 & \times & 6 & 24 & 36 & 24 & 6 \\
 & & 4 & 16 & 24 & 16 & 4 \\
 & & 1 & 4 & 6 & 4 & 1
 \end{array}$$

The normalising constant. The coefficients must sum to 1 or the image changes brightness. Here:

$$\sum = (1 + 4 + 6 + 4 + 1) \times (1 + 4 + 6 + 4 + 1) = 16 \times 16 = 256$$

Mask = (1/256) × the array above

Note that the same construction gives the 3×3 Gaussian from the row 1 2 1, with a normalising sum of $4 \times 4 = 16$ — which is the familiar $(1/16)[1 \ 2 \ 1; 2 \ 4 \ 2; 1 \ 2 \ 1]$.

DRAWN FOR THE ANSWER

Box / mean	Gaussian	Laplacian	Sharpen																																				
<table><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	1	1	1	1	1	<table><tr><td>1</td><td>2</td><td>1</td></tr><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>2</td><td>1</td></tr></table>	1	2	1	2	4	2	1	2	1	<table><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>-4</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr></table>	0	1	0	1	-4	1	0	1	0	<table><tr><td>0</td><td>-1</td><td>0</td></tr><tr><td>-1</td><td>5</td><td>-1</td></tr><tr><td>0</td><td>-1</td><td>0</td></tr></table>	0	-1	0	-1	5	-1	0	-1	0
1	1	1																																					
1	1	1																																					
1	1	1																																					
1	2	1																																					
2	4	2																																					
1	2	1																																					
0	1	0																																					
1	-4	1																																					
0	1	0																																					
0	-1	0																																					
-1	5	-1																																					
0	-1	0																																					
1/9	1/16	sum 0	sum 1																																				
smooths, blurs edges too	smooths, weights the centre	a pure second derivative	the original plus its edges																																				

the fastest check there is:

coefficients sum to 1 → a smoothing or sharpening mask, brightness is preserved
coefficients sum to 0 → a derivative mask, flat regions give zero response

The masks worth knowing by heart, and the one check that tells you at a glance which kind you have written.

Q-2
(c)

Using the concept of image reconstruction using n-bit planes, find the new values of the pixels by combining the 8th bit plane and the 7th bit plane for the bold pixels (167 and 111) of the given image A. C02

03

FROM THE PAPER

$$A = \begin{bmatrix} \mathbf{167} & 133 & \mathbf{111} \\ 144 & 140 & 135 \\ 159 & 154 & 148 \end{bmatrix}$$

Fig.3

Fig.3 — the image A. The two bold entries, 167 and 111, are the pixels the question is about.

SOLUTION

Step 1 — write both values in 8-bit binary. The 8th bit plane is the MSB (weight 128) and the 7th is the next one (weight 64).

bit :	8	7	6	5	4	3	2	1
weight :	128	64	32	16	8	4	2	1

167 =	1	0	1	0	0	1	1	1
check:	128 + 32 + 4 + 2 + 1 = 167 OK							

111 =	0	1	1	0	1	1	1	1
check:	64 + 32 + 8 + 4 + 2 + 1 = 111 OK							

Step 2 — keep only planes 8 and 7, and zero everything else.

167 :	bit8 = 1	→	128
	bit7 = 0	→	0
	new value = 128 + 0 = 128		
111 :	bit8 = 0	→	0
	bit7 = 1	→	64
	new value = 0 + 64 = 64		

167 → 128 · 111 → 64

Comment. Two planes can only generate four distinct outputs — 0, 64, 128 and 192 — so the reconstruction is a coarse four-level image. The overall shapes survive, because most of a pixel's magnitude lives in its top bits; all the fine

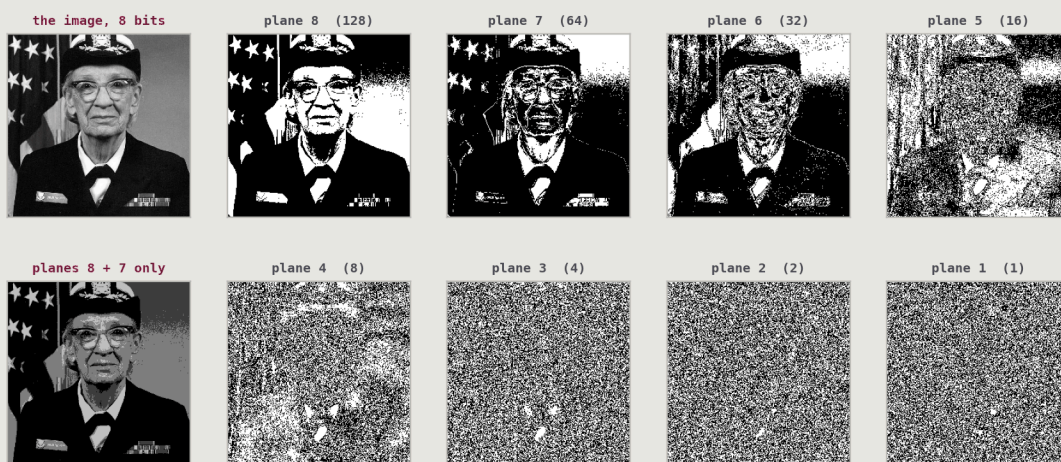
shading is gone, because that is what the lower planes carried. This is exactly the observation the lecture makes about the higher planes holding the visually significant data.

WATCH THE PLANE NUMBERING

“8th bit plane” here means the **most significant bit**, counting from 1 at the LSB. Some books number the planes 0 to 7 instead, in which case the same two planes would be called the 7th and 6th. Say which convention you are using in one line — it takes a second and removes the ambiguity.

COMPUTED · TOOLS/MAKE-FIGURES.PY

The top planes carry the picture; the bottom ones carry noise. Two planes give four grey levels: 0, 64, 128, 192



What the question is really about, on a real photograph: the top planes carry the picture and the bottom ones carry noise.

Q-2
(d)

Consider the 2-bit image of size 5×5 shown in Fig.4. Calculate the mean using the two formulas discussed in the class and compare the answers.

03

FROM THE PAPER

0	0	1	1	2
1	2	3	0	1
3	3	2	2	0
2	3	1	0	0
1	1	3	2	2

Fig.4

Fig.4 — the 2-bit 5×5 image, 25 pixels with values 0 to 3.

SOLUTION

0	0	1	1	2
1	2	3	0	1
3	3	2	2	0
2	3	1	0	0
1	1	3	2	2

MN = 25, values 0..3 (2 bits, L = 4)

Formula 1 — direct spatial average. Add every pixel and divide by how many there are:

$$m = \left(\frac{1}{MN} \right) \sum_x \sum_y f(x, y)$$

row sums : $0 + 0 + 1 + 1 + 2 = 4$

$1 + 2 + 3 + 0 + 1 = 7$

$3 + 3 + 2 + 2 + 0 = 10$

$2 + 3 + 1 + 0 + 0 = 6$

$1 + 1 + 3 + 2 + 2 = 9$

total = 36

$$m = \frac{36}{25} = 1.44$$

Formula 2 — from the histogram. First count each gray level:

R_I	COUNT N_I	$P(R_I) = N_I/25$	$R_I \times P(R_I)$
0	6	0.24	0.00
1	7	0.28	0.28
2	7	0.28	0.56
3	5	0.20	0.60
total		1.00	1.44

$$m = \sum r_i p(r_i) = 0 + 0.28 + 0.56 + 0.60 = 1.44$$

Both formulas give $m = 1.44$ — and they must

The comparison, which is what the question is asking for. The two are the *same* calculation organised differently. Formula 1 walks the image pixel by pixel; Formula 2 groups the identical pixels first and weights each distinct value by how often it occurs. Grouping cannot change a sum, so the answers are identical by construction.

Why bother with the second one, then? Because it costs one pass over the **histogram** rather than one pass over the **image**. For a 4×4 toy that is no saving; for a 4096×4096 image with 256 gray levels it is 256 terms instead of 16.7 million. And once the histogram exists, every other statistic is nearly free:

$$\begin{aligned}
 \text{variance} &= \sum (r_i - m)^2 p(r_i) \\
 &= (0 - 1.44)^2(0.24) + (1 - 1.44)^2(0.28) \\
 &\quad + (2 - 1.44)^2(0.28) + (3 - 1.44)^2(0.20) \\
 &= 0.4977 + 0.0542 + 0.0878 + 0.4867 \\
 &= 1.126 \quad \text{so} \quad \sigma = 1.061
 \end{aligned}$$

Mean measures **brightness**; variance measures **contrast**. Together they are the pair used for the local adaptive enhancement in Lecture 19.

Q-3 Consider the image segment shown in Fig.5. (i) Let $V = \{0,2,3\}$. Find the pixel relationship for $N_4(p)$, $N_D(p)$ and $N_8(p)$ (write only coordinates). (ii) Let $V = \{0,3\}$. Compute the lengths of the shortest 4-, 8- and m-paths between p and q. If a particular path does not exist between these two points, explain why. 04

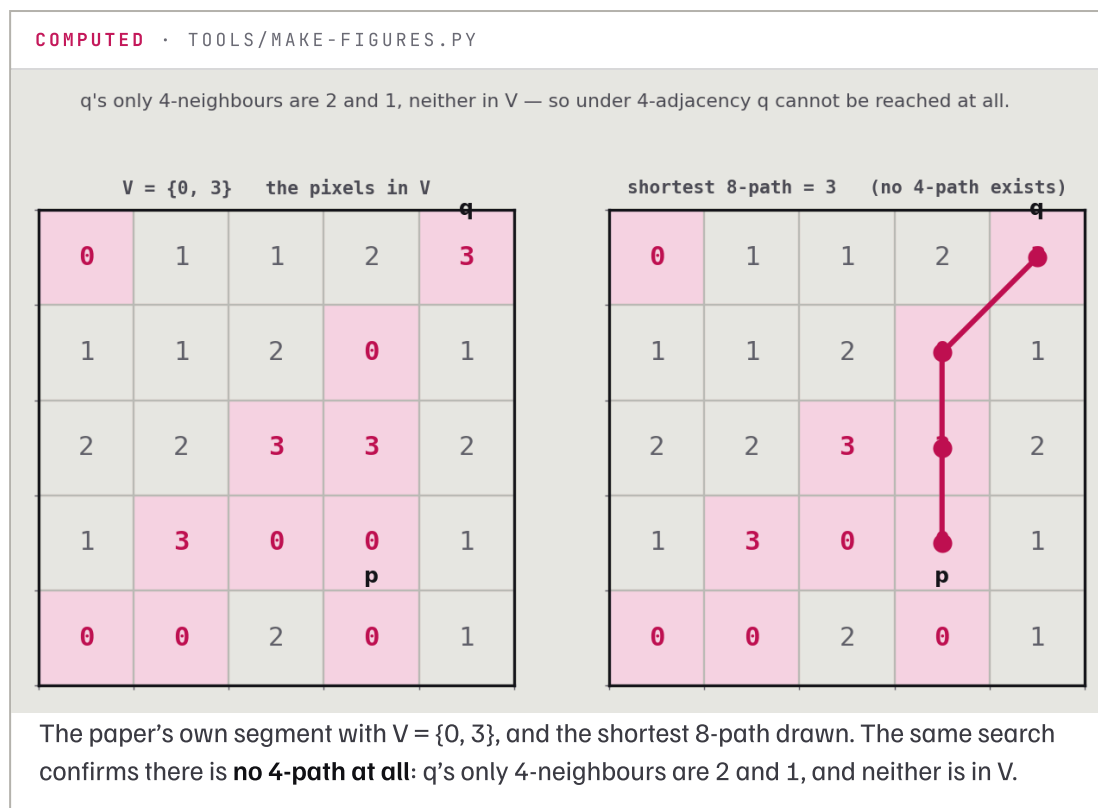
FROM THE PAPER

0 (0,0)	1	1	2	3 (q)
1	1	2	0	1
2	2	3	3	2
1	3	0	0 (p)	1
0	0	2	0	1

Fig.5

Fig.5 — the image segment. p is at (3,3) and q at (0,4), both marked on the scan.

SOLUTION



(i) Neighbourhoods of $p = (3,3)$. Written as (row, column):

$$N_4(p) = (2, 3), (4, 3), (3, 2), (3, 4)$$

$$N_D(p) = (2, 2), (2, 4), (4, 2), (4, 4)$$

$$\begin{aligned} N_8(p) &= N_4(p) \cup N_D(p) \\ &= (2, 2), (2, 3), (2, 4), (3, 2), (3, 4), (4, 2), (4, 3), (4, 4) \end{aligned}$$

If the question wants only those neighbours whose *values* lie in $V = \{0, 2, 3\}$, check each:

$$\begin{array}{llll} (2, 3) = 3 & \text{in } V & (4, 3) = 0 & \text{in } V & (3, 2) = 0 & \text{in } V & (3, 4) = 1 & \text{NOT} \\ (2, 2) = 3 & \text{in } V & (2, 4) = 2 & \text{in } V & (4, 2) = 2 & \text{in } V & (4, 4) = 1 & \text{NOT} \end{array}$$

so seven of the eight qualify; (3,4) and (4,4)
do not.

(ii) Paths from $p = (3,3)$ to $q = (0,4)$, with $V = \{0, 3\}$. Mark which pixels belong to V :

```

col:    0    1    2    3    4
row 0:  *    .    .    .    *    q
row 1:  .    .    .    *    .
row 2:  .    .    *    *    .
row 3:  .    *    *    *    .    p
row 4:  *    *    .    *    .

```

* = value is 0 or 3 (in V)

Shortest 4-path: it does not exist.

Look at $q = (0,4)$. Its only 4-neighbours inside the image are:

$(0,3) = 2 \rightarrow \text{NOT in } V$

$(1,4) = 1 \rightarrow \text{NOT in } V$

No 4-path exists

The reason, which is the mark: a 4-path may only take horizontal and vertical steps between pixels that are *both* in V . Neither of q 's two 4-neighbours is in V , so q is completely isolated under 4-adjacency — nothing can reach it, however long the route.

Shortest 8-path. Diagonals now count, and q does have a diagonal neighbour in V :

$(0,4)$ and $(1,3)$ are diagonal neighbours, and $(1,3) = 0$ is in V

path : $(3,3) \rightarrow (2,3) \rightarrow (1,3) \rightarrow (0,4)$

p vertical vertical diagonal

values : $0 \rightarrow 3 \rightarrow 0 \rightarrow 3$ all in V

length = 3 steps

Is 3 the minimum? The chessboard distance sets the floor:

$$D_8(p, q) = \max(|3 - 0|, |3 - 4|) = \max(3, 1) = 3$$

Shortest 8-path = 3, and it is optimal

Shortest m-path. Take the same route and test its one diagonal step against the m-adjacency rule — a diagonal is allowed only if $N_4(p) \cap N_4(q)$ contains **no** pixel from V .

steps $(3,3) \rightarrow (2,3)$ and $(2,3) \rightarrow (1,3)$ are 4-adjacent, so allowed at once.

step $(1,3) \rightarrow (0,4)$ is diagonal. Test it:

$N_4(1,3) = \{ (0,3), (2,3), (1,2), (1,4) \}$

$N_4(0,4) = \{ (0,3), (1,4) \}$

common = $\{ (0,3), (1,4) \}$

values : $(0,3) = 2 \rightarrow$ not in V

$(1,4) = 1 \rightarrow$ not in V

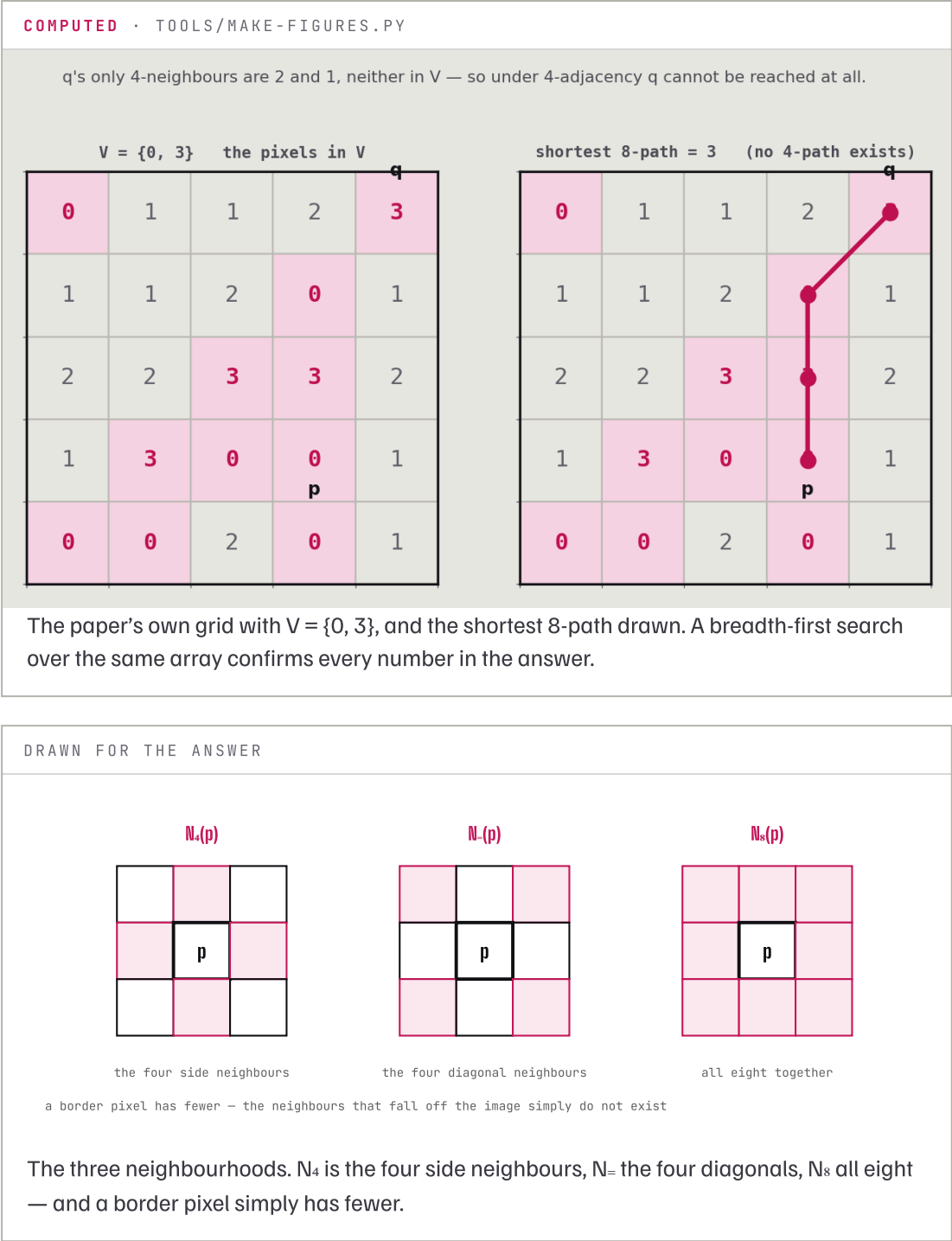
neither is in V , so the diagonal step IS allowed.

length = 3 steps

Shortest m-path = 3

THE COMMENT THAT COMPLETES THE ANSWER

4-path: none · 8-path: 3 · m-path: 3. The m-path here happens to equal the 8-path, because the one diagonal it needed had no route round it. That is the general rule: **m-adjacency only removes a diagonal link when a way round already exists**, so an m-path is never shorter than the 8-path and often the same length — what it guarantees is that the path is **unique**.



FROM THE PAPER

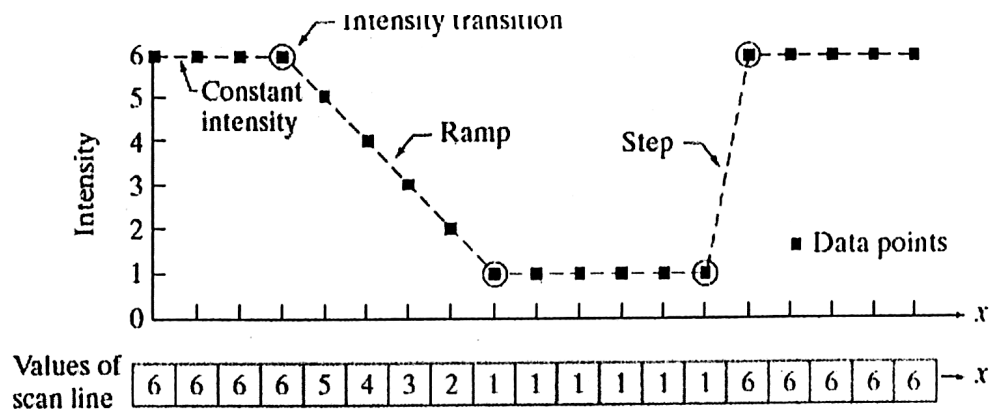


Fig.6

Fig.6 — the gray-level profile, with the constant region, the ramp and the step labelled, and the values of the scan line written underneath.

SOLUTION

values of the scan line:

```

index : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
value : 6 6 6 6 5 4 3 2 1 1 1 1 1 1 6 6 6 6 6
      |-----| |----ramp----| |--constant--| |step| |const|

```

First derivative $f'(x) = f(x+1) - f(x)$, computed from index 1 through 17:

```

index : 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
f'(x) : 0 0 -1 -1 -1 -1 -1 0 0 0 0 0 5 0 0 0 0

```

Second derivative $f''(x) = f(x+1) + f(x-1) - 2f(x)$, over the same range:

```

index : 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
f''(x): 0 0 -1 0 0 0 0 1 0 0 0 0 5 -5 0 0 0

```

Sample computations, to show the working:

```

f'(3)  = f(4) - f(3)           = 5 - 6       = -1
f'(13) = f(14) - f(13)        = 6 - 1       = 5

f''(3)  = f(4) + f(2) - 2f(3)   = 5 + 6 - 12 = -1
f''(8)  = f(9) + f(7) - 2f(8)   = 1 + 2 - 2   = 1
f''(13) = f(14) + f(12) - 2f(13) = 6 + 1 - 2   = 5
f''(14) = f(15) + f(13) - 2f(14) = 6 + 1 - 12 = -5

```

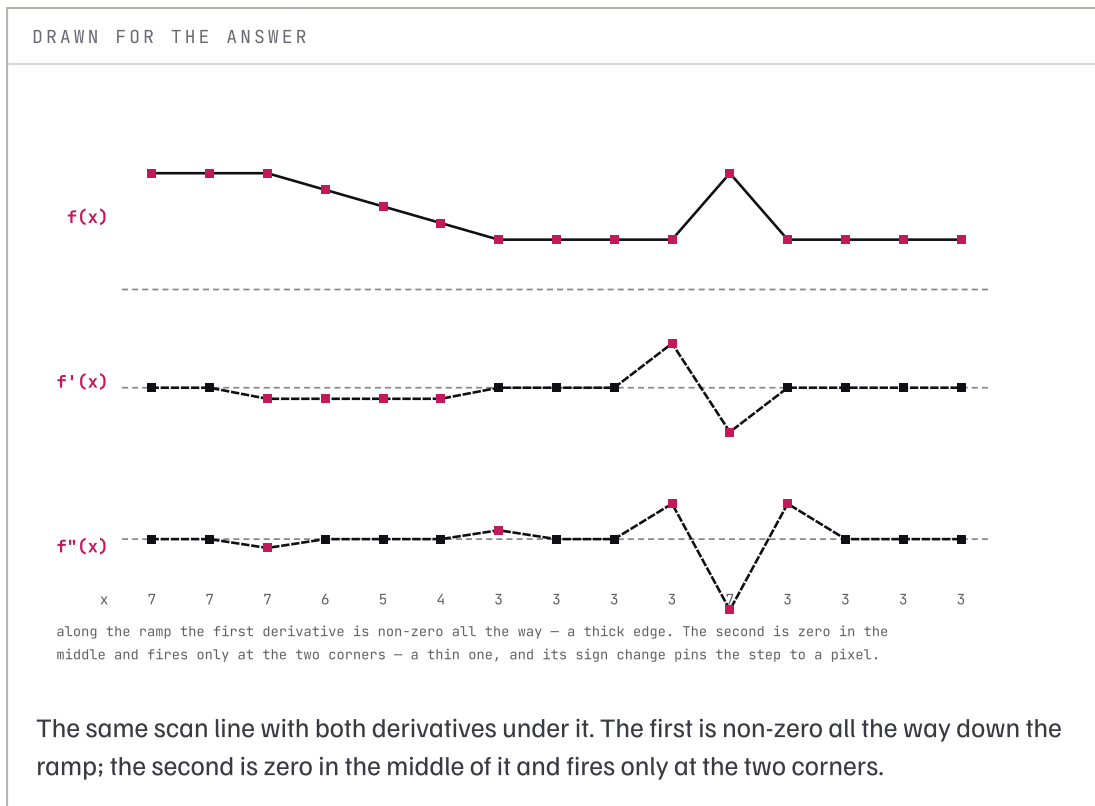
Comments — this is where the marks are.

REGION	FIRST DERIVATIVE	SECOND DERIVATIVE
Constant intensity	Zero, as required	Zero, as required
Along the ramp (6 down to 1)	Non-zero throughout — a steady -1 all the way down, so it produces a thick response spanning the entire ramp	Zero in the middle of the ramp; non-zero only at its onset (-1) and its end ($+1$), so a thin response marking just the two corners
At the step (1 up to 6)	A single spike of $+5$	A double response: $+5$ immediately followed by -5 , of <i>opposite</i> sign
Zero crossing	None	The point between the $+5$ and the -5 — it locates the edge to the exact pixel

The conclusion to write out:

- **First derivatives produce thicker edges** and respond throughout a region of gradual change.
- **Second derivatives have a much stronger response to fine detail** — isolated points and thin lines — and they are zero along ramps of constant slope.
- The second derivative's **double response of opposite sign** at a step, and the zero crossing between, is what makes it useful for locating edges precisely.
- Both are zero in constant regions, which is why every derivative mask has coefficients that sum to zero.

That is exactly why the **Laplacian** — a second derivative — is the standard sharpening filter, and the **gradient** — a first derivative — is the standard edge detector.



Q-4
(a)

Suppose that a 3-bit image ($L = 8$) has the intensity distribution shown in Table I. It is desired to transform the original histogram to the specified histogram of Table II. What will be the actual histogram after the histogram matching operation? Find the transformation between r and z . *Note: do not round histogram-equalized values to the nearest integer until the final mapping table, to avoid ambiguities.* Also draw the graphs for (i) $P_z(z_q)$ vs z_q and (ii) $P_s(s_k)$ vs s_k .

05

s_k . C02

SOLUTION

TABLE I									TABLE II										
rk	:	0	1	2	3	4	5	6	7	zq	:	0	1	2	3	4	5	6	7
nk	:	10	5	5	11	5	0	3	6	Pz(zq):		5	5	5	10	10	5	5	0
total = 45 in both																			

Step 1 — equalise the INPUT histogram. $s_k = (L-1) \times$ running sum of p_r , so $s_k = 7 \times (\text{cumulative } n)/45$:

R_k	N_k	CUMULATIVE	$S_k = 7 \times \text{CUM} / 45$
0	10	10	1.5556

R_K	N_K	CUMULATIVE	$S_K = 7 \times \text{CUM} / 45$
1	5	15	2.3333
2	5	20	3.1111
3	11	31	4.8222
4	5	36	5.6000
5	0	36	5.6000
6	3	39	6.0667
7	6	45	7.0000

Step 2 — equalise the SPECIFIED histogram. $G(z_q) = 7 \times (\text{cumulative } P_z) / 45$:

z_q	P_z	CUMULATIVE	$G(z_q)$
0	5	5	0.7778
1	5	10	1.5556
2	5	15	2.3333
3	10	25	3.8889
4	10	35	5.4444
5	5	40	6.2222
6	5	45	7.0000
7	0	45	7.0000

Step 3 — for each s_k , find the z_q whose $G(z_q)$ is closest. This is the mapping the question calls “the transformation between r and z ”.

R_K	S_K	NEAREST $G(z_q)$	$\rightarrow z_q$
0	1.5556	1.5556 (exact)	1
1	2.3333	2.3333 (exact)	2
2	3.1111	tie: 2.3333 and 3.8889 are both 0.7778 away	2
3	4.8222	5.4444 (0.6222 away)	4
4	5.6000	5.4444 (0.1556 away)	4

R_k	S_k	NEAREST $G(Z_q)$	$\rightarrow Z_q$
5	5.6000	5.4444	4
6	6.0667	6.2222 (0.1556 away)	5
7	7.0000	7.0000 (exact)	6

The tie at $r = 2$ matters. $s_2 = 3.1111$ sits exactly halfway between $G(2)$ and $G(3)$. The convention is to **choose the smallest z_q** , so $z = 2$. That is precisely the ambiguity the note in the question is warning about — rounding s_k to 3 first would have hidden it.

Step 4 — the actual output histogram. Pour each n_k into its mapped z :

$$z = 1 \leftarrow r_0 = 10$$
$$z = 2 \leftarrow r_1 + r_2 = 5 + 5 = 10$$
$$z = 4 \leftarrow r_3 + r_4 + r_5 = 11 + 5 + 0 = 16$$
$$z = 5 \leftarrow r_6 = 3$$
$$z = 6 \leftarrow r_7 = 6$$
$$z = 0, 3, 7 = 0$$
$$\text{total} = 10 + 10 + 16 + 3 + 6 = 45 \text{ OK}$$

Z_q	0	1	2	3	4	5	6	7
SPECIFIED	5	5	5	10	10	5	5	0
ACTUAL	0	10	10	0	16	3	6	0

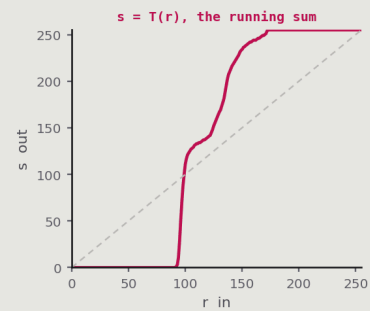
Step 5 — the two graphs.

COMPUTED · TOOLS/MAKE-FIGURES.PY

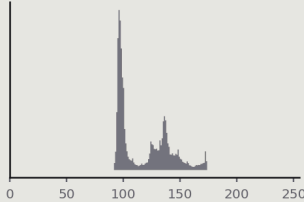
before everything between 92 and 173



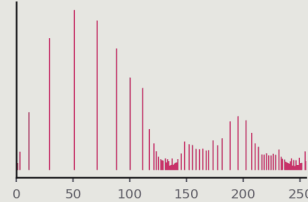
after equalisation



crowded into a third of the range



spread across all of it

**What actually happens**

Equalisation cannot split a grey level — every pixel of one value must go to the same new value. So the bars are moved and merged, never divided, and the result is spread rather than flat.

std before 22.5 → std after 70.8

The two graphs the question asks for, drawn from the real numbers.

THE COMMENT THAT FINISHES THE ANSWER

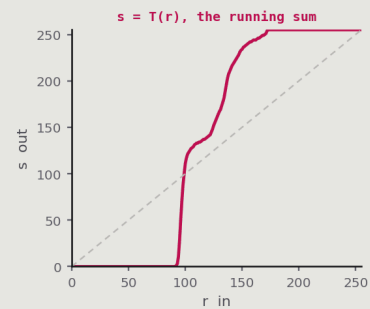
The actual histogram **only approximates** the specified one, and it always will. Matching cannot split a gray level — all 11 pixels of value 3 must go to the *same* z — so the bars can only be moved and merged, never divided. In the continuous case the match would be exact; in the discrete case you get the nearest achievable shape, and saying so is worth a mark.

COMPUTED · TOOLS/MAKE-FIGURES.PY

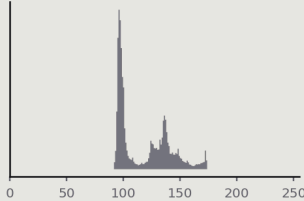
before everything between 92 and 173



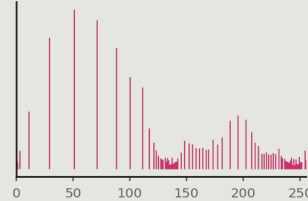
after equalisation



crowded into a third of the range



spread across all of it

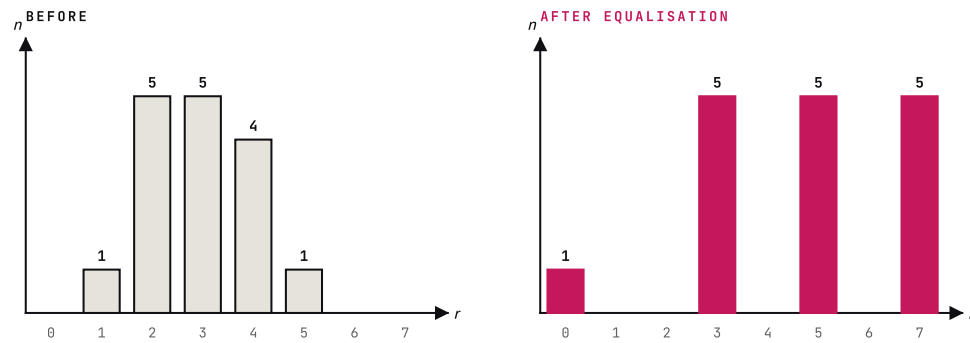
**What actually happens**

Equalisation cannot split a grey level — every pixel of one value must go to the same new value. So the bars are moved and merged, never divided, and the result is spread rather than flat.

std before 22.5 → std after 70.8

Equalisation on a real image, with both histograms and the transfer curve. Note the output is spread, not flat — exactly the point the last mark is for.

DRAWN FOR THE ANSWER



equalisation cannot split a grey level – every pixel of one value must go to the same new value,
so the bars are moved and merged, never divided. The result is spread out, not flat.

What equalisation actually does: the bars are moved apart and merged, never split. That is why the result is spread rather than flat.

Q-4
(b)

Suppose an image has a probability density function given in Fig.7(a). We would like to modify it so that it has the probability density function given in Fig.7(b). Derive the transformation function $g(f)$ that will accomplish this. For simplicity, assume both the original and the modified image can take on gray levels in the continuous range $(0, 255)$. Also find the value of the derived function $g(f)$ at $f = 0$ and $f = 255$. C02

04

FROM THE PAPER

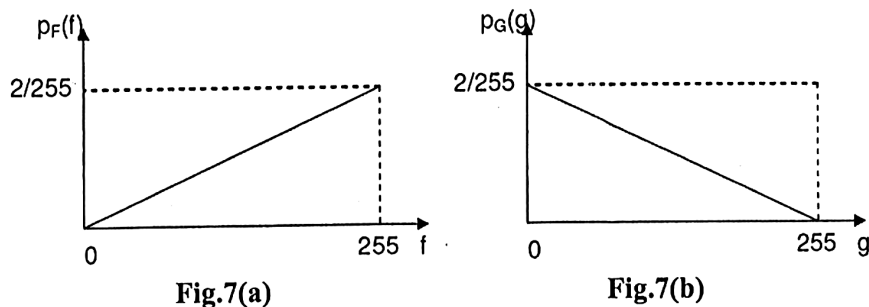


Fig.7(a) and Fig.7(b) — the density the image has, and the one it is to be given.

SOLUTION

Read the two PDFs off the figures.

Fig.7(a): a straight line rising from 0 at $f = 0$ to $2/255$ at $f = 255$.

$$p_F(f) = \frac{2f}{255^2} \quad \text{for } 0 \leq f \leq 255$$

check : $\int_0^{255} \frac{2f}{255^2} df = \left[\frac{f^2}{255^2} \right] = 1 \text{ OK}$

Fig.7(b): a straight line falling from 2/255 at $g = 0$ to 0 at $g = 255$.

$$p_G(g) = \frac{2(255 - g)}{255^2} \quad \text{for } 0 \leq g \leq 255$$

check : $\int_0^{255} \frac{2(255 - g)}{255^2} dg$

$$= \left(\frac{2}{255^2} \right) \left[255g - \frac{g^2}{2} \right] = \left(\frac{2}{255^2} \right) \left(\frac{255^2}{2} \right) = 1 \text{ OK}$$

Step 1 — equalise the input.

$$\begin{aligned} s = T(f) &= 255 \int_0^f p_F(w) dw \\ &= 255 \int_0^f \frac{2w}{255^2} dw \\ &= 255 \times \left[\frac{w^2}{255^2} \right]_0^f \\ &= \frac{f^2}{255} \end{aligned}$$

Step 2 — equalise the target.

$$\begin{aligned} G(g) &= 255 \int_0^g p_G(t) dt \\ &= 255 \times \left(\frac{2}{255^2} \right) \left[255t - \frac{t^2}{2} \right]_0^g \\ &= \left(\frac{2}{255} \right) \left(255g - \frac{g^2}{2} \right) \\ &= \frac{510g - g^2}{255} \end{aligned}$$

Step 3 — set $G(g) = T(f)$ and solve for g .

$$\begin{aligned}\frac{510g - g^2}{255} &= \frac{f^2}{255} \\ 510g - g^2 &= f^2 \\ g^2 - 510g + f^2 &= 0\end{aligned}$$

A quadratic in g . Apply the formula:

$$\begin{aligned}g &= [510 \pm \sqrt{510^2 - 4 f^2}] / 2 \\ &= 255 \pm \sqrt{255^2 - f^2}\end{aligned}$$

Which root? The plus sign gives $g \geq 255$ for every f , which is outside the range.

Take the minus sign:

$$\begin{aligned}g(f) &= 255 - \sqrt{255^2 - f^2} \\ &= 255 - \sqrt{65025 - f^2}\end{aligned}$$

$$g(f) = 255 - \sqrt{65025 - f^2}$$

The two values asked for:

$$\begin{aligned}f = 0: \quad g &= 255 - \sqrt{65025 - 0} = 255 - 255 = 0 \\ f = 255: \quad g &= 255 - \sqrt{65025 - 65025} = 255 - 0 = 255\end{aligned}$$

$$g(0) = 0 \quad \cdot \quad g(255) = 255$$

Both endpoints map to themselves, which is exactly what a monotonic transformation covering the full range must do — a useful check that the algebra is right.

What the transformation actually does. Try the midpoint:

$$\begin{aligned}f = 128 &\rightarrow g = 255 - \sqrt{65025 - 16384} \\ &= 255 - \sqrt{48641} \\ &= 255 - 220.5 \\ &= 34.5\end{aligned}$$

Mid-grey is pushed a long way **down**, to 34.5. That is the right direction: the input PDF rises with f , so the image is mostly bright; the target PDF falls with g , so it wants mostly dark. The transformation obliges by compressing the bright end heavily and stretching the dark end — and the curve is the lower-left quarter of a circle of radius 255.

EC341 · DIGITAL IMAGE PROCESSING · PAPERS WORKED BY HAND AND CHECKED NUMERICALLY
THE PREDICTED PAPER IS OUR OWN READING OF THE PATTERN — IT IS NOT AN OFFICIAL ANYTHING, AND
IT IS NOT A LEAK
BACK TO TU CHAHIYE · THE ORIGINAL SCANS · ALL THE PDFS